

1. Reactive Programming vs Blocking — ต่างกันอย่างไร?

- **Blocking Programming** คือรูปแบบการทำงานที่โปรแกรมจะต้องรอให้การทำงานหนึ่งเสร็จสิ้นก่อน จึงจะสามารถทำงานต่อได้ เช่น การเรียก API แล้วต้องรอ Response กลับมาก่อน ทำให้ Thread ที่กำลังรอไม่สามารถนำไปทำงานอื่นได้ในเวลานั้น

- **Reactive Programming** เป็นรูปแบบการเขียนโปรแกรมที่เน้นการทำงานแบบ Asynchronous และ Non-blocking โดยโปรแกรมไม่จำเป็นต้องหยุดรอผลลัพธ์ แต่สามารถดำเนินการอื่นต่อได้ และเมื่อข้อมูลพร้อมจึงส่งข้อมูลกลับมา โดยใน Spring WebFlux จะใช้ Mono และ Flux เป็นตัวแทนของข้อมูลที่สามารถส่งเข้ามาในอนาคต

2. Mono vs Flux — ใช้กรณีไหน และทำไม?

- **Mono**: ใช้สำหรับแทนข้อมูลที่มีจำนวน 0 หรือ 1 ค่า เหมาะกับกรณีที่ต้องการข้อมูล Product เพียงรายการเดียว เช่น การค้นหา Product ตาม ID การสร้าง Product หรือการคำนวณราคาหลังส่วนลด

ตัวอย่างในโปรเจกต์:

```
Mono<Product> findById(String id)
Mono<Product> save(Product product)
Mono<Double> getDiscountedPrice(String id)
```

- **Flux**: ใช้สำหรับแทนข้อมูลที่มีจำนวน 0 ถึงหลายค่า เหมาะกับกรณีที่ต้องการข้อมูลหลายรายการ เช่น การค้นหา Product ทั้งหมด หรือการค้นหา Product ตาม Category

ตัวอย่างในโปรเจกต์:

```
Flux<Product> findAll()
Flux<Product> findByCategory(String category)
```

ดังนั้นการเลือกใช้ Mono หรือ Flux จะขึ้นอยู่กับจำนวนข้อมูลที่ Method นั้นต้องการส่งกลับ โดย Mono เหมาะกับข้อมูลหนึ่งรายการ ส่วน Flux เหมาะกับข้อมูลหลายรายการ

3. WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()

- **WebClient**: เป็น HTTP Client ของ Spring WebFlux ที่ใช้สำหรับเรียก REST API และรองรับการทำงานแบบ Reactive และ Non-blocking โดยในโปรเจกต์นี้ใช้ WebClient สำหรับเรียก Endpoint ของ Product ที่สร้างขึ้น

ตัวอย่าง Method Chain:

```
client.get()
    .uri("/products/{id}", id)
    .retrieve()
    .bodyToMono(Product.class);
```

แต่ละ Method มีหน้าที่ดังนี้:

- .get() ใช้กำหนด HTTP Method เป็น GET
- .uri() ใช้กำหนด URL หรือ Endpoint ที่ต้องการเรียก และสามารถส่งค่า Path Variable ได้
- .retrieve() ใช้ส่ง Request และเตรียมรับ Response จาก Server
- .bodyToMono(Product.class) ใช้แปลงข้อมูล Response Body ให้เป็น Mono<Product> ในกรณีที่ Response มีข้อมูล Product เพียงหนึ่งรายการ
- .bodyToFlux(Product.class) ใช้แปลงข้อมูล Response Body ให้เป็น Flux<Product> ในกรณีที่ Response มี Product หลายรายการ

4. Code ของ Repository, Service, Controller พร้อม comment อธิบาย?

4.1 ProductRepository ทำหน้าที่จัดเก็บและค้นหาข้อมูล Product โดยใน Lab นี้ใช้ ConcurrentHashMap เป็น In-memory Storage แทนการเชื่อมต่อ Database เพื่อเน้นการฝึกใช้งาน Mono และ Flux

- **findById()** ใช้สำหรับค้นหา Product จาก id โดยถ้าพบข้อมูลจะคืนค่าเป็น Mono<Product> และถ้าไม่พบจะคืน Mono.empty()

```

public Mono<Product> findById(String id) {
    Product product = store.get(id);
    if (product == null) {
        return Mono.empty();
    }
    return Mono.just(product);
}

```

- **findAll()** ใช้สำหรับค้นหา Product ทั้งหมด โดย store.values() จะได้ Collection ของ Product และใช้ Flux.fromIterable() เพื่อแปลงข้อมูลให้เป็น Flux

```

public Flux<Product> findAll() {
    return Flux.fromIterable(store.values());
}

```

- **save()** ใช้สำหรับบันทึก Product ลงใน store โดยใช้ id เป็น Key และคืน Product ที่บันทึกกลับมาในรูปแบบ Mono

```

public Mono<Product> save(Product product) {
    store.put(product.getId(), product);
    return Mono.just(product);
}

```

- **deleteById()** ใช้สำหรับลบ Product ตาม ID จาก store และคืนค่าเป็น Mono<Void>

```

public Mono<Void> deleteById(String id) {
    store.remove(id);
    return Mono.empty();
}

```

- **findByCategory()** ใช้สำหรับค้นหา Product ตาม Category โดยใช้ filter() เพื่อเลือกเฉพาะ Product ที่มี Category ตรงกับค่าที่กำหนด

```

public Flux<Product> findByCategory(String category) {
    return findAll()
        .filter(p -> p.getCategory().equalsIgnoreCase(category));
}

```

4.2 ProductService ทำหน้าที่เป็นชั้นกลางระหว่าง Controller และ Repository โดยรับผิดชอบ Business Logic ของระบบ เช่น การตรวจสอบ Product การสร้าง ID และการคำนวณราคาหลังส่วนลด

- **getById()** เรียกข้อมูลจาก Repository และใช้ switchIfEmpty() เพื่อจัดการกรณีที่ไม่มี Product

```

public Mono<Product> getById(String id) {
    return repository.findById(id)
        .switchIfEmpty(
            Mono.error(
                new RuntimeException(
                    "Product not found: " + id)));
}

```

- **getAll()** ใช้สำหรับเรียก Product ทั้งหมดจาก Repository

```
public Flux<Product> getAll() {
    return repository.findAll();
}
```

- **save()** ใช้สำหรับบันทึก Product และหาก Product ไม่มี ID จะสร้าง ID ใหม่ด้วย UUID

```
public Mono<Product> save(Product product) {
    if (product.getId() == null) {
        product.setId(UUID.randomUUID().toString());
    }
    return repository.save(product);
}
```

- **delete()** ใช้สำหรับลบ Product ตาม ID

```
public Mono<Void> delete(String id) {
    return repository.deleteById(id);
}
```

- **getByCategory()** ใช้สำหรับค้นหา Product ตาม Category

```
public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}
```

- **getDiscountedPrice()** ใช้สำหรับคำนวณราคาหลังส่วนลด โดยใช้ map() เพื่อแปลงข้อมูลจาก Product เป็นราคาหลัง Discount

```
public Mono<Double> getDiscountedPrice(String id) {
    return repository.findById(id)
        .map(p -> p.getDiscountedPrice());
}
```

4.3 ProductController ทำหน้าที่รับ HTTP Request จาก Client และเรียกใช้งาน Service ที่เกี่ยวข้อง โดย Controller จะคืนค่าเป็น Mono หรือ Flux เพื่อให้ทำงานแบบ Reactive และ Non-blocking

- **GET Product ตาม ID** ใช้สำหรับค้นหา Product หนึ่งรายการจาก ID จึงใช้ Mono<Product>

```
@GetMapping("/{id}")
public Mono<Product> getById(@PathVariable String id) {
    return service.getById(id);
}
```

- **GET Product ทั้งหมด** ใช้สำหรับเรียก Product ทั้งหมด จึงใช้ Flux<Product>

```
@GetMapping
public Flux<Product> getAll() {
    return service.getAll();
}
```

- **POST สร้าง Product** ใช้สำหรับรับข้อมูล Product จาก Request Body และบันทึกลงระบบ

```
@PostMapping
public Mono<Product> create(@RequestBody Product product) {
    return service.save(product);
}
```

```
}
```

- **DELETE Product** ใช้สำหรับลบ Product ตาม ID และคืนค่าเป็น Mono<Void>

```
@DeleteMapping("/{id}")
public Mono<Void> delete(@PathVariable String id) {
    return service.delete(id);
}
```

- **GET Product ตาม Category** ใช้สำหรับค้นหา Product ตาม Category และสามารถคืนข้อมูลได้หลายรายการ จึงใช้ Flux<Product>

```
@GetMapping("/category/{category}")
public Flux<Product> getByCategory(
    @PathVariable String category) {
    return service.getByCategory(category);
}
```

- **GET ราคาหลัง Discount** ใช้สำหรับแสดงราคาของ Product หลังคำนวณส่วนลด โดยคืนค่าเป็น Mono<Double>

```
@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(
    @PathVariable String id) {
    return service.getDiscountedPrice(id);
}
```